

Sistemas Operativos

Un SO es un programa o conjunto de programas que administran los recursos físicos (HW), el software y la interfaz de usuario. Además, se administra a sí mismo y se encarga de proveer seguridad en el sistema, asegurando que los programas que permite ejecutar hagan lo que deben hacer y no otras cosas que puedan dañar a la computadora. También permiten la gestión de usuarios

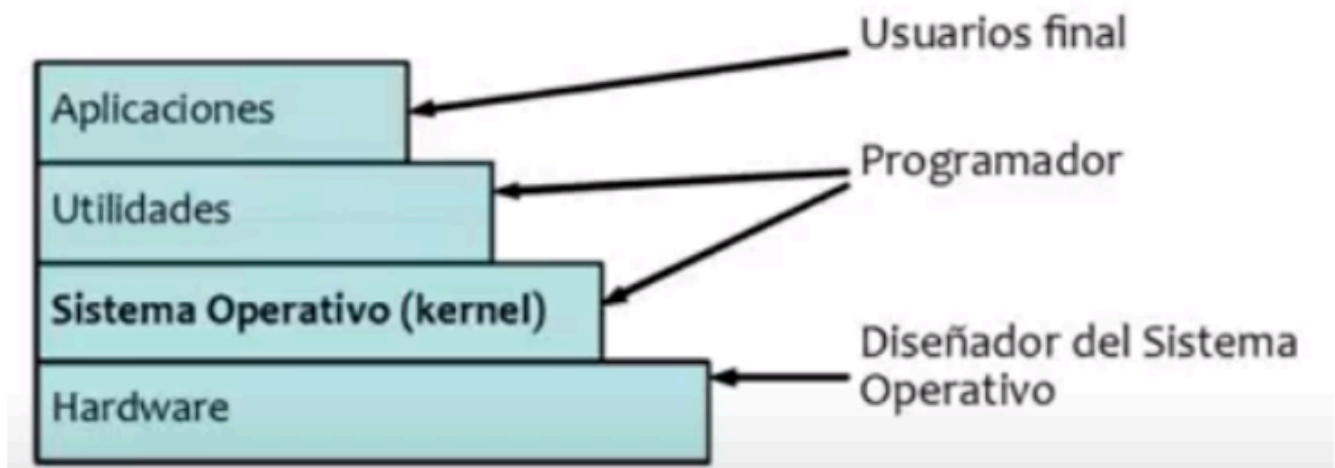
Funciones de los SOs

- Administrar la ejecución de programas.
- Proveer una interfaz para usuarios y programadores.
- Administrar los recursos de HW y SW.
- Administrar los dispositivos de E/S.
- Administrar los archivos.
- Administrar la comunicación entre programas.
- Asignar recursos.
- Brindar protección y seguridad.
- Administrarse a sí mismo.
- Ser interfaz de los dispositivos.

Capas de una computadora

Todo SO tiene un core (núcleo) donde están desarrolladas las funcionalidades básicas. Pero, el sistema no sería útil si no tuviera aplicaciones, una interfaz de usuario y demás funciones utilitarias (compilador, consola, debugger). Entonces tenemos muchas capas:

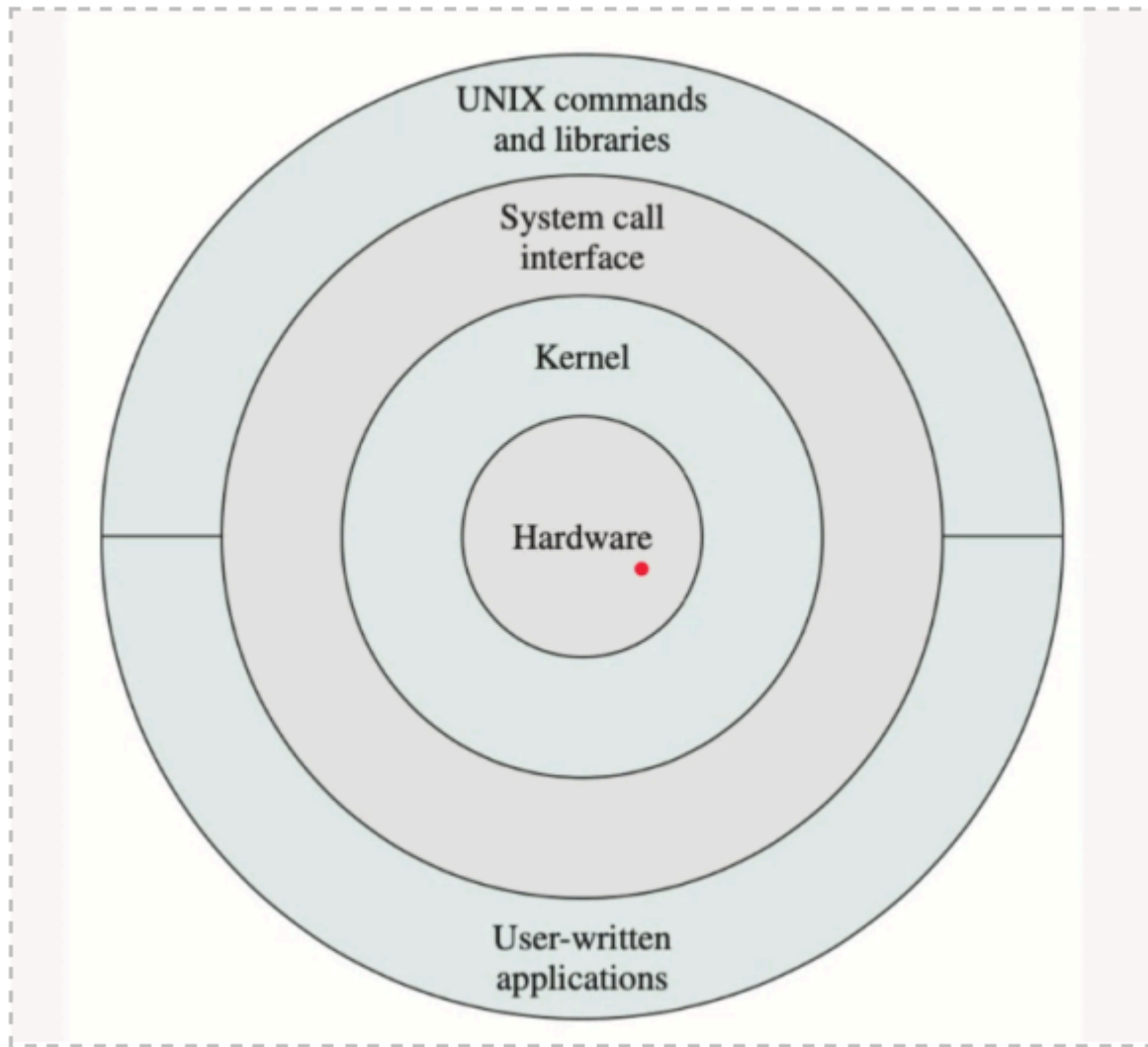
- **Kernel:** contiene las funciones principales de un SO. Es su núcleo. Gestiona los recursos hardware del sistema y suministra la funcionalidad básica del SO. A su vez, el kernel nos permite la sincronización y comunicación entre procesos.
- **Distribuciones:** Son sistemas operativos basados en el núcleo que, además, incluyen determinados paquetes de software con aplicaciones para usos específicos.
 - Aplicaciones: destinadas al uso por parte del usuario final. Estas aplicaciones o programas, a la hora de usar el hardware, tiene que pasar por el SO. Los programas no hablan directamente con el hardware, sino que el SO está de intermediario.
 - Utilidades: recursos usados por los programadores para interactuar con el SO y el HW.



Evolución de los SOs

- *Monoprogramados*: permite la ejecución de un sólo programa a la vez y dispones de todos los recursos disponibles para este programa.
 - Procesamiento en serie
 - Sistemas en lotes sencillos: permite seleccionar una serie de programas y definir que se ejecute uno después del otro.
- *Multiprogramados*: permite a muchos programas ejecutar de manera **concurrente**. El SO debe asumir un rol de administrador ya que debe encargarse de la asignación de recursos a los programas que quieran ejecutar.
 - Sistemas en lotes multiprogramados: permiten que, mientras está esperando un evento de un dispositivo de E/S, pueda haber otro programa ejecutando.
 - Sistemas de tiempo compartido: permite a muchos usuarios ejecutar de manera concurrente. La tarea de administración es mucho más compleja dado que no todos los usuarios disponen de los mismos recursos.

Llamadas al sistema (SYSCALLS)



Las **syscalls** son funciones incluidas en el kernel del SO mediante las cuales un programa puede solicitar servicios al SO. Estos servicios implican típicamente acceder al HW, o acceder a programas a los que sólo el SO tiene acceso. Es decir, el único modo que tienen las aplicaciones de hablar con el HW es a través de llamadas al sistema. Las mismas son accesibles a través de un lenguaje de programación porque son simplemente funciones, por ejemplo:

- Operaciones sobre archivos: `open()`, `read()`, `write()`
- Operaciones sobre procesos: `fork()`, `exit()`, `kill()`
- Manipulación de dispositivos: `release()`, `eject()`
- Otras: `time()`, `sem_wait()`

Cada SO tiene definidas sus propias llamadas al sistema.

Wrappers

Los **wrappers** son funciones que pretenden ser estándares a todos los sistemas operativos y que envuelven a las llamadas al sistema. Es decir, es el wrapper el que se encarga de hacer la llamada al sistema. Los wrappers aportan:

- Portabilidad: permiten ejecutar mi programa en cualquier SO.
- Simplicidad: Me ahorro detalles del SO que no me interesan.
- Eficiencia.

Hacer uso directo de las funciones de cada SO me otorga mayor precisión pero menor simplicidad y portabilidad.

Modos de ejecución

Los SOs manejan un "anillo de protección" (que habitualmente es entre 0 y 3) y, de acuerdo al nivel en el que estemos, podemos ejecutar ciertos tipos de instrucciones. Habitualmente se habla únicamente de dos modos de ejecución:

- Kernel(0): usado por el SO. Puede ejecutar cualquier tipo de instrucciones. Accedemos al modo kernel mediante interrupciones o syscalls.
- Usuario (3): sólo puede ejecutar instrucciones no privilegiadas. Los programas se ejecutan únicamente en modo usuario. Si un proceso quiere hacer uso de una instrucción privilegiada debe realizar una llamada al sistema.

Cambio de modo (Mode switch)

Definición: Es el cambio entre los modos de ejecución posibles.

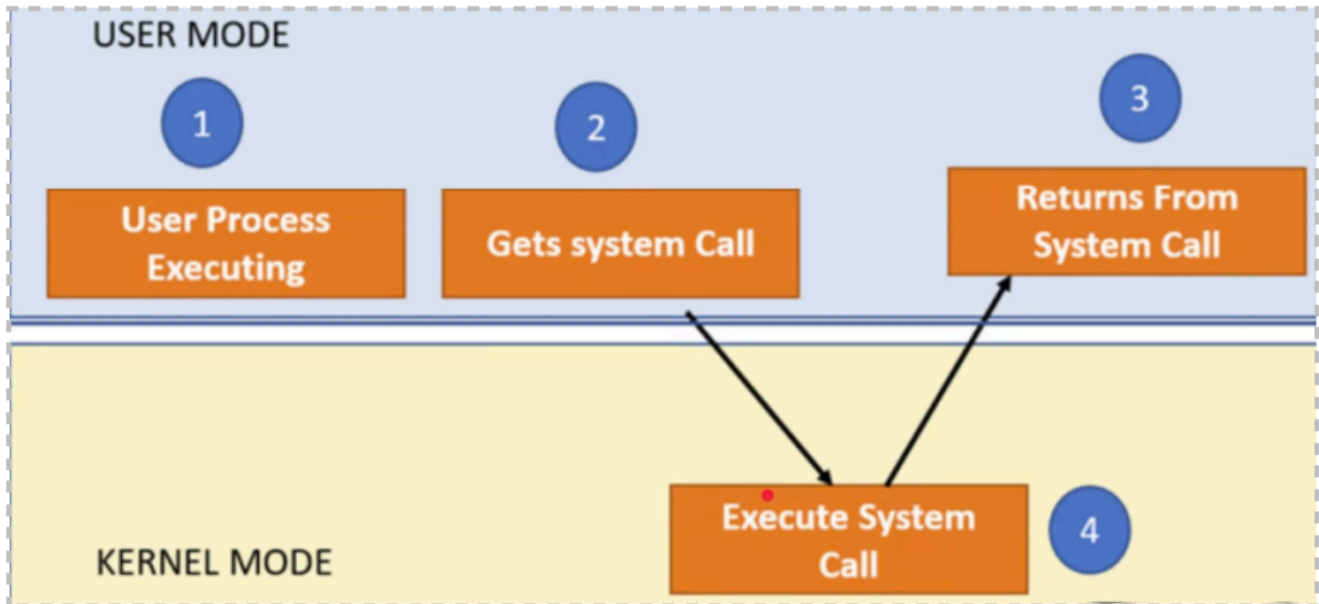
- Las aplicaciones siempre ejecutan en modo usuario.
- El SO siempre ejecuta en modo kernel

Al ejecutar distintos procesos se producen cambio de modo constantemente. Nunca puede ocurrir que ejecute una aplicación e inmediatamente otra porque es el SO quien decide cuál sigue. Debe haber siempre un cambio a modo kernel. Para que deje de ejecutar una aplicación pueden ocurrir dos cosas:

- Una *interrupción* (suponiendo que se decida atenderla). Para atenderla deberá ejecutar el SO, entonces cambiará a modo kernel.
- Cuando la aplicación realice una *llamada al sistema*.

¿Cuándo se produce?

Un usuario **no** puede cambiar el modo de ejecución. El cambio se produce cuando un programa realiza una llamada al sistema. Todos los programas comienzan en modo usuario. Cuando se recibe la syscall empieza a ejecutar el sistema operativo y hace lo que tiene que hacer. Después se devuelve el resultado al proceso, que continúa ejecutando en modo usuario.



¿Puede un programa ejecutar una instrucción privilegiada?

No, el programa no puede ejecutar la instrucción. Se produce una interrupción y se llama al SO. El SO le manda una señal al programa para que finalice (un *segmentatio fault*) o toma alguna acción.